Greedy Two-Pointer Approach

Yes. Your code is using the **greedy + two-pointer** approach. The key idea is:

> **Always consider the heaviest remaining person. Try to pair them with the lightest remaining person. If even these two cannot fit, then the heaviest person must go alone.**

This gives the minimum number of boats.

---

1. First understand the problem

Each boat can carry:

- At most **2 people**
- Their combined weight must be **`<= limit`**

For example:

```
people = [1, 2, 2, 3]
limit = 3
```

We want the minimum boats.

One optimal arrangement:

```
1 + 2 = 3  → boat
2          → boat
3          → boat

Answer = 3
```

2. Why do we sort?

After sorting:

```
[1, 2, 2, 3]
 ↑        ↑
left     right
lightest heaviest
```

We maintain two pointers:

```
left  → lightest remaining person
right → heaviest remaining person
```

Why specifically the **heaviest** person?

Because the heaviest person is the hardest person to accommodate.

---

3. The most important greedy observation

Suppose:

```
people = [1, 2, 3, 4]
limit = 5
```

Look at the heaviest person:

```
4
```

The best possible partner for **`4`** is the **lightest person**:

```
1 + 4 = 5
```

They fit.

So:

```
1 + 4 → one boat
```

Why is this safe?

Because if **`4`** can be paired with **`1`**, then pairing **`4`** with anyone heavier than **`1`** is no better.

For example:

```
2 + 4 = 6 ❌
3 + 4 = 7 ❌
```

So the lightest person is our **best chance** to save a boat.

---

4. What if even the lightest + heaviest don't fit?

Suppose:

```
people = [1, 2, 3, 4]
limit = 4
```

Take:

```
lightest = 1
heaviest = 4
```

Check:

```
1 + 4 = 5
```

That's greater than `4`.

So they cannot share a boat.

But notice something important:

Since `1` is the **lightest** person, everyone else is:

```
1, 2, 3, 4
   ↑  ↑
   all >= 1
```

Therefore:

```
2 + 4 >= 1 + 4 > limit
3 + 4 >= 1 + 4 > limit
```

So `4` **cannot pair with anybody**.

Therefore:

```
4 → alone in one boat
```

We simply move:

```
C++
right--;
```

This is the critical greedy logic.

---

5. Complete example

Consider:

```
people = [3, 2, 2, 1]
limit = 3
```

Step 1: Sort

```
[1, 2, 2, 3]
 ↑        ↑
 L        R
```

Check:

```
1 + 3 = 4 > 3
```

So `3` cannot pair with anyone.

```
3 → boat
```

Move **`right`**.

```
[1, 2, 2, 3]
 ↑     ↑
```

```
 L     R
```

Step 2

Now:

```
1 + 2 = 3
```

They fit.

So:

```
1 + 2 → boat
```

Move both pointers:

```
left++;
right--;
```

Remaining:

```
[1, 2, 2, 3]
     ↑
    L/R
```

Step 3

Remaining person:

```
2
```

They need one boat.

Answer:

```
3 boats
```

6. Why this gives the minimum?

Think about the heaviest remaining person **`people[right]`**.

There are only two possibilities:

Case 1: Lightest + heaviest fit

```
C++
people[left] + people[right] <= limit
```

Then pairing them is optimal.

Why?

Because if the heaviest person can fit with the lightest, there's no reason to leave the lightest for somebody else while the heaviest takes a boat.

We can safely use:

```
lightest + heaviest
```

and remove both.

Case 2: Lightest + heaviest don't fit

```
C++
people[left] + people[right] > limit
```

Then the heaviest person cannot pair with **anyone**.

So:

```
heaviest → alone
```

Remove only the heaviest.

---

7. Your comments can be improved

Your current comment:

```
C++
//if pairing two highest and lowest weight is less than the limit then one boat can take them
```

There aren't two highest/lowest. There is one **lightest** and one **heaviest**.

A more accurate comment is:

```
C++
// Try to pair the heaviest person with the lightest person.
// If they fit, putting them together saves one boat.
```

And:

```
C++
// If even the lightest person cannot fit with the heaviest,
// then nobody else can fit either, so the heaviest person
// must take a boat alone.
```

8. Final clean code

```
C++
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {

        // Sort so that:
        // left  -> lightest remaining person
        // right -> heaviest remaining person
        sort(people.begin(), people.end());

        int left = 0;
        int right = people.size() - 1;

        int boats = 0;

        while (left <= right) {

            // Try to pair the heaviest person with the lightest person.
            // If they fit, both can share one boat.
            if (people[left] + people[right] <= limit) {
                left++;
                right--;
            }
            else {
                // Even the lightest person cannot fit with the heaviest.
                // Therefore, the heaviest person cannot fit with anyone
                // and must take a boat alone.
                right--;
            }

            // In both cases, one boat is used.
            boats++;
        }

        return boats;
    }
};
```

Complexity

Sorting dominates:

```
Sorting: O(n log n)
Two pointers: O(n)

Total: O(n log n)
Space: O(1) extra
```

The one-line intuition to remember

> **Take the heaviest person. If they can fit with the lightest person, pair them; otherwise, send the heaviest alone.**

This is the same kind of **greedy two-pointer pattern** you can reuse in problems where you need to minimize groups/pairs under a capacity constraint.